

Pool Game: Concurrent Implementation Report

Assignment 01 - PCD Course

Academic Year 2025-2026

Abstract

This report presents a concurrent implementation of the Pool Game, a 2D board game where two players (human and bot) control balls to push smaller balls into holes. The document provides a detailed analysis of concurrency challenges, design architecture, behavioral specifications using Petri Nets, performance evaluation, and formal verification using JPF (Java Path Finder). The concurrent implementation demonstrates significant improvements over a sequential baseline while effectively exploiting available processor cores through multithreading and task-based approaches.

Contents

1	Problem Analysis	3
1.1	Problem Description	3
1.2	Concurrency-Related Challenges	3
1.2.1	Computational Complexity	3
1.2.2	Temporal Constraints	3
1.2.3	Synchronization Requirements	3
1.2.4	Data Races and Race Conditions	4
1.3	Task-based Related Challenges	4
1.3.1	Computational Complexity	4
1.3.2	Temporal Constraints	5
1.3.3	Synchronization Requirements	5
2	Design and Architecture	7
2.1	Architectural Overview for Multithreading	7
2.1.1	Component Description	7
2.2	Synchronization Strategy	8
2.2.1	Monitor-Based Synchronization	8
2.2.2	Bot Monitor	9
2.2.3	Rendering Synchronization (RenderSync)	9
2.3	Thread Lifecycle	9
2.4	Architectural Overview for Task-Based Approach	10
2.4.1	Component Descriptions	10
2.5	Synchronization Strategy	10

3	Behavior Specification: Petri Nets	10
3.1	High-Level Game Flow Net	10
3.2	Update Thread Behavior Net	11
3.3	Bot Thread Behavior Net	11
4	Performance Evaluation	12
4.1	Experimental Setup	12
4.1.1	Test Configurations	12
4.1.2	Performance Metrics	12
4.2	Expected Performance Improvements	12
4.2.1	Sequential Baseline Issues	12
4.2.2	Concurrent Improvements	12
4.2.3	Task-based Improvements	13
4.3	Overhead	13
4.3.1	Multithread overhead	13
4.3.2	Task-Based overhead	13
5	Formal Verification with JPF	13
5.1	Java Path Finder Integration	13
5.1.1	JPF Configuration	13
5.2	Safety Properties Verified	14
5.2.1	1. Mutual Exclusion (No Data Races)	14
5.2.2	2. Deadlock Freedom	14
5.2.3	3. Liveness (Game Termination)	14
5.3	Race Condition Detection	15
5.3.1	Synchronized Field Accesses	15
5.3.2	Temporal Assertions	15
5.4	Model Checking Results	15
5.4.1	Test Configuration: MainJPF	15
6	Conclusion	16
A	Code Listings	17
A.1	GameController Main Loop	17

1 Problem Analysis

1.1 Problem Description

The Pool Game is a two-player billiard-like game on a 2D board with the following characteristics:

- **Dynamic Objects:** Multiple balls (typically ranging from 2 to 4500) moving and interacting on the board
- **Physics Simulation:** Elastic collisions, friction forces, and momentum transfer between bodies
- **Players:** Two players (human controlled via keyboard, bot controlled autonomously) with scoring systems
- **Game Objective:** Insert small balls into holes at board corners; winner is determined by score or by eliminating opponent's ball
- **Performance Bottlenecks:** The sequential version of Pool exhibits several performance bottlenecks that motivate concurrent design:

1.2 Concurrency-Related Challenges

1.2.1 Computational Complexity

- **Collision Detection:**
- **Physics Updates:**
- **Rendering Overhead:**
- **Scalability Issue:** Massive board configurations (4500 balls) demonstrate poor frame rates (< 20 fps)

1.2.2 Temporal Constraints

- Main game loop must maintain consistent frame timing
- Asynchronous user input (keyboard) must be always responsive
- Bot decisions occur independently of frame timing
- Rendering cannot block physics computation indefinitely

1.2.3 Synchronization Requirements

- Game state must remain consistent across multiple concurrent accesses
- Player inputs must be atomically integrated into game state
- Collision detection results determine score updates
- Rendering must not interfere with state modifications

1.2.4 Data Races and Race Conditions

Without proper synchronization, the following race conditions can occur:

- **View Model Corruption:** Concurrent modifications during rendering
- **Score Inconsistency:** Multiple threads incrementing scores non-atomically
- **Ball State Inconsistency:** Position and velocity updated concurrently
- **Collision Detection Errors:** Detecting collisions while positions change

1.3 Task-based Related Challenges

The task-based concurrent implementation using the Executor Framework introduces a different set of challenges compared to the thread-per-entity model. The Pool implementation uses work-stealing task decomposition via `WorkerPool` to parallelize physics updates and collision detection.

1.3.1 Computational Complexity

Task-based parallelism in the Pool implementation must address several complexity-related challenges:

- **Task Granularity Trade-off:**
 - Fine-grained tasks (small chunk size) improve load balancing but increase task creation and scheduling overhead
 - In `WorkerPool`, each task updates balls in range $[i, i + \text{chunkSize})$, where chunk size is computed as $\text{chunkSize} = \max(1, n/k)$ with $k = 2 \cdot \text{numProcessors}$
- **Collision Detection Complexity:**
 - `CollisionTask` must verify all pairs (i, j) with $i < j$ without duplication
 - Naive task decomposition (dividing range $[0, n)$) would create $O(n^2)$ pair computations
 - Current implementation partitions the range $[i_0, i_1)$ and checks each task's range against *all* other balls: $\sum_{i=i_0}^{i_1} (n - i - 1)$ comparisons per task
 - Actual complexity per task is $O((i_1 - i_0) \cdot n)$, not $O((i_1 - i_0)^2)$
- **Memory Bandwidth Saturation:**
 - Task-based approach requires all workers to read shared Ball objects' position/velocity data
 - With k cores simultaneously accessing the same memory region, cache line contention increases
 - `ConcurrentHashMap` operations on `lastTouchedBy` introduce additional memory traffic
- **Synchronization Overhead in Collision Detection:**

- Each collision check acquires locks on two Ball objects using `synchronized(first)` then `synchronized(second)`
- Lock ordering (by `System.identityHashCode()`) prevents deadlock but adds comparison overhead

1.3.2 Temporal Constraints

Task-based parallelism introduces timing-related constraints that differ from the thread-per-entity model:

- **Barrier Synchronization Latency:**
 - In `WorkerPool`, both `parallelBallUpdate()` and `parallelCollisionDetection()` use `invokeAll()`, which is a synchronization barrier
 - All tasks must complete before control returns to the `GameLoop`
 - Frame time = $\max(\text{task duration}) + \text{overhead}$, not $\sum(\text{task durations}) / k$
- **Task Queue Delay:**
 - Created tasks are submitted to the `ExecutorService` queue
 - If all workers are busy, new tasks wait in queue, delaying their execution
 - With a fixed thread pool of size k , tasks may experience queueing delay proportional to queue depth
 - This adds non-deterministic latency to each game frame
- **Future.get() Blocking Behavior:**
 - The main `GameLoop` thread calls `f.get()` (code in class `WorkerPool`) on all futures from `invokeAll()`
 - This is a blocking operation—the `GameLoop` cannot proceed until *all* tasks complete
 - Any uncaught exception in a task will be re-thrown at `get()`, potentially crashing the loop

1.3.3 Synchronization Requirements

Task-based parallelism requires careful synchronization to ensure memory consistency and avoid data races:

- **Nested Lock Acquisition in Collision Detection:**
 - `CollisionTask.call()` must synchronize on two Ball objects to resolve collisions atomically:

```
1 synchronized (first) {  
2     synchronized (second) {  
3         // Atomic collision resolution  
4         Ball.resolveCollision(a, b);  
5     }  
6 }
```

Listing 1: Ordered Lock Acquisition

- The ordering by `System.identityHashCode()` prevents deadlock but adds computational cost
- If two tasks try to resolve the same collision pair with different order, one will block the other
- **ConcurrentHashMap for lastTouchedBy:**
 - Multiple `CollisionTask` instances concurrently update `lastTouchedBy` map via `put()` calls
 - Using a non-concurrent map would cause data races
 - `ConcurrentHashMap` uses bucket-level locking (segment locking), reducing contention vs. a single global lock
 - However, repeated `put()` calls on the same `Ball` key still incur synchronization overhead
- **Read-Write Visibility:**
 - After `invokeAll().get()`, all field writes in tasks are visible to the main `GameLoop` (happens-before guarantee)
 - The Java Memory Model ensures that synchronization actions (lock acquire/release) create memory barriers
 - Without these barriers, updates to `Ball.vel` and `Ball.pos` made by one task might not be visible to the next phase
- **Task Isolation:**
 - `BallUpdateTask` accesses `board` (the `Board` object) to query boundary constraints
 - This creates a shared dependency: if `Board` is modified during task execution, physics updates may read stale boundary data
 - Current implementation assumes `Board` state (bounds, player positions) is immutable during a phase
 - If asynchronous modifications occur (e.g., player ball movement), visibility must be explicitly synchronized
- **Exception Handling and Task Failure:**
 - If a task throws an unchecked exception, `f.get()` will re-throw it via `ExecutionException`
 - The `GameLoop` must handle `InterruptedException` (task thread interrupted) and `ExecutionException` (task computation failed)
 - Without proper handling, the entire game loop may crash
 - Current implementation in `GameLoop.run()` catches both exceptions and breaks the loop gracefully

2 Design and Architecture

2.1 Architectural Overview for Multithreading

The concurrent Pool implementation adopts an MVC (Model-View-Controller) architectural pattern with multiple active components:

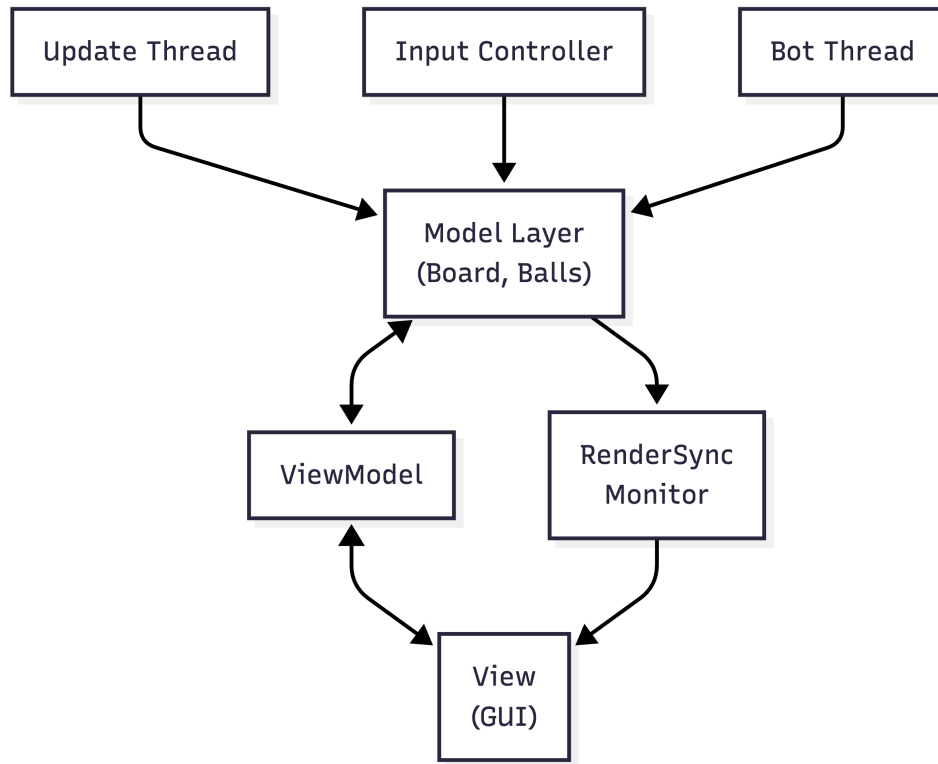


Figure 1: A descriptive caption for your image.

2.1.1 Component Description

Model Layer

The Model layer encapsulates the game state and physics engine:

- **Board:** Central synchronized object maintaining:
 - List of balls with positions and velocities
 - Player ball states (human and bot)
 - Scores for both players
 - Game-over state
 - Ball-to-player touch tracking
- **Ball:** Represents physical entities with mass, radius, position, and velocity
- **Boundary:** Defines board limits for collision detection
- **Monitor (Board):** Synchronized access ensures atomic game state updates

Controller Layer

The Controller comprises two main active components:

- **Update Thread:** Maintains the main game loop
 - Periodically updates board physics (elapsed time-based)
 - Synchronizes with View for rendering
 - Maintains frame rate statistics
 - Checks game-over conditions
- **Bot Thread:** Autonomous player agent
 - Monitors bot ball velocity
 - Generates random kick actions when ball is idle ($v < 0.05$)
 - Waits on bot monitor for notifications
 - Respects game-over state

View Layer

- **ViewFrame:** Swing JFrame managing GUI rendering
- **ViewModel:** Passive data holder with synchronized access
- **RenderSync:** Monitor pattern implementation for synchronous rendering
- **Keyboard Input Handler:** Asynchronously captures user input

2.2 Synchronization Strategy

2.2.1 Monitor-Based Synchronization

The implementation uses Java monitors (synchronized blocks/methods) rather than low-level primitives:

```
1 public synchronized void updateState(long dt) {
2     // Atomic physics computation
3     for (Ball b : balls) {
4         b.updateState(dt, this);
5     }
6     // Atomic collision detection
7     for (int i = 0; i < balls.size(); i++) {
8         for (int j = i + 1; j < balls.size(); j++) {
9             Ball.resolveCollision(balls.get(i), balls.get(j));
10        }
11    }
12 }
```

Listing 2: Board Synchronized Methods

2.2.2 Bot Monitor

A dedicated monitor controls bot behavior synchronization:

```
1 synchronized (board.getBotMonitor()) {
2     while (isRunning()) {
3         var p2 = board.getPlayer2();
4         if (p2.getVel().abs() < 0.05) {
5             // Generate kick action
6         } else {
7             try {
8                 board.getBotMonitor().wait();
9             } catch (InterruptedException e) {
10                // Handle interruption
11            }
12        }
13    }
14 }
```

Listing 3: Bot Thread Synchronization

2.2.3 Rendering Synchronization (RenderSync)

Custom monitor ensures rendering completion before continuing state updates:

```
1 public void render() {
2     long nf = sync.nextFrameToRender();
3     panel.repaint();
4     if (!SwingUtilities.isEventDispatchThread()) {
5         try {
6             sync.waitForFrameRendered(nf);
7         } catch (InterruptedException ex) {
8             ex.printStackTrace();
9         }
10    }
11 }
```

Listing 4: RenderSync Implementation

2.3 Thread Lifecycle

The game execution follows this sequence:

1. **Initialization:** Board configured with balls, boundaries, players
2. **Thread Creation:** Update and Bot threads spawned as daemon threads
3. **Game Execution:**
 - Update thread cycles main game loop
 - Bot thread waits on monitor, activates on velocity threshold
 - Both threads access shared Board state via synchronization
4. **Termination:** Game-over flag set, threads interrupted and stopped

2.4 Architectural Overview for Task-Based Approach

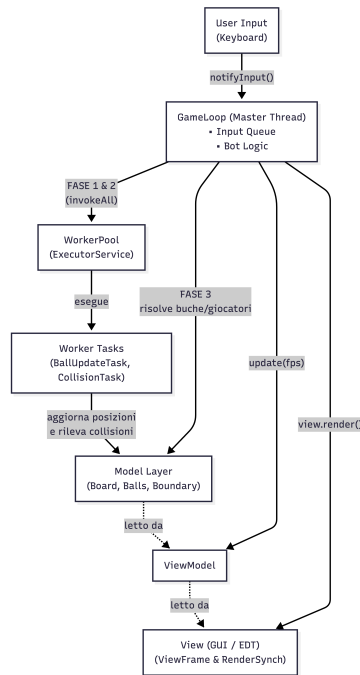


Figure 2: A descriptive caption for your image.

2.4.1 Component Descriptions

Task Decomposition

2.5 Synchronization Strategy

3 Behavior Specification: Petri Nets

3.1 High-Level Game Flow Net

The overall game behavior is modeled using a Petri Net capturing the major state transitions:

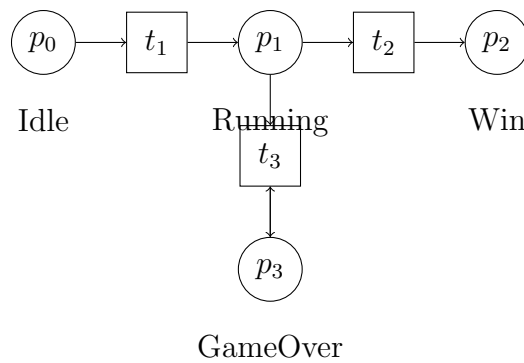


Figure 3: Game State Petri Net: p_0 (Idle) $\xrightarrow{t_1} p_1$ (Running) $\xrightarrow{t_2} p_2$ (Win/End) or $\xrightarrow{t_3} p_3$ (GameOver)

- p_0 : Initial idle state
- t_1 : Game initialization and thread startup
- p_1 : Game running state with two active threads
- t_2 : All balls removed or game-over condition detected
- p_2 : Final winning state
- p_3 : Player ball in hole (alternative end)
- t_3 : Continuous looping in game-over state

3.2 Update Thread Behavior Net

Detailed behavior of the Update thread cycle:

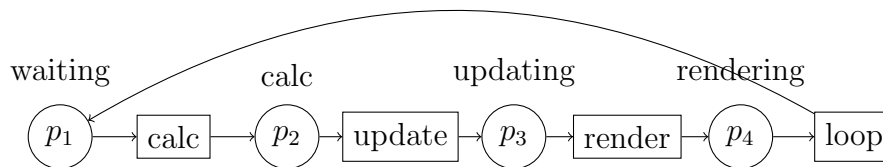


Figure 4: Update Thread Loop: Sequential phases of elapsed time calculation, board state update, view model update, and rendering

3.3 Bot Thread Behavior Net

Behavior of the Bot thread with conditional transitions:

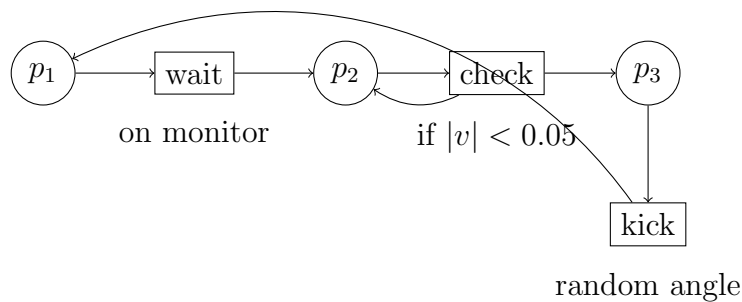


Figure 5: Bot Thread: Waits on monitor, checks velocity, kicks when idle

The key guard condition is $|v| < 0.05$, representing the velocity threshold below which the bot is permitted to kick. This threshold prevents the bot from kicking while its ball is already moving at significant speed.

4 Performance Evaluation

4.1 Experimental Setup

4.1.1 Test Configurations

Three board configurations are tested:

Configuration	Small Balls	Expected Complexity
Minimal	2	
Large	400	
Massive	4,500	

Table 1: Board Configurations and Collision Detection Complexity

4.1.2 Performance Metrics

- **Frame Rate (FPS):** Frames rendered per second
- **Update Latency:** Time to compute one physics update cycle
- **CPU Utilization:** Percentage of available cores used
- **Throughput:** Simulation steps per second
- **Speedup:** Concurrent vs. sequential frame rate ratio

4.2 Expected Performance Improvements

4.2.1 Sequential Baseline Issues

The sequential implementation exhibits:

- Minimal/Large: fps (acceptable)
- Massive: 2 fps (unacceptable frame rate degradation)

4.2.2 Concurrent Improvements

Update Thread Parallelization:

- Potential to parallelize ball state updates (independent operations)
- Potential to parallelize collision detection
- Rendering no longer blocks physics computation

Bot Thread Independence:

- Bot decisions computed asynchronously
- Reduces decision latency from main game loop
- Enables responsive user input even during intensive physics

Expected Speedup Formula:

$$S = \frac{T_{seq}}{T_{conc}} = \frac{f_{seq}}{f_{conc}}$$

For massive configuration with parallel collision detection over k cores:

$$S \approx 1 + \frac{(O(n^2) - O(n))}{k}$$

With $n = 4,500$ balls and $k = 4$ cores:

$$S \approx 1 + \frac{10.1M - 4,500}{4} \approx 2.5\text{-}2.8\times$$

// Note: Actual speedup may be lower due to synchronization overhead and non-parallelizable code sections.

4.2.3 Task-based Improvements**4.3 Overhead****4.3.1 Multithread overhead**

The monitor-based synchronization introduces overhead:

- **Lock Contention:** Both Update and Bot threads compete for Board lock
- **Context Switching:** Multiple threads increase scheduler overhead
- **Cache Coherency:** Shared data requires cache invalidation
- **Rendering Synchronization:** RenderSync monitor introduces artificial barriers

The overhead is typically offset by parallelization gains in configurations with significant collision detection workload (400+ balls).

4.3.2 Task-Based overhead**5 Formal Verification with JPF****5.1 Java Path Finder Integration**

JPF (Java Path Finder) is a model checker developed at NASA for verifying Java programs. The Pool implementation integrates JPF verification through configuration and test drivers.

5.1.1 JPF Configuration

Key aspects of JPF application to Pool:

```

1 target=pcd.assignment01.controller.MainJPF
2 listener = gov.nasa.jpf.listener.PropertyListener

```

Listing 5: JPF Verification Approach

5.2 Safety Properties Verified

5.2.1 1. Mutual Exclusion (No Data Races)

Property: Board state modifications are atomic and do not interleave.

Verification Method:

- JPF tracks synchronized block boundaries
- No two threads can execute synchronized Board methods concurrently
- All field accesses to mutable state (balls, scores) occur within synchronization

Critical Section Analysis:

```

1 public synchronized void updateState(long dt) {
2     // CRITICAL SECTION START
3     for (Ball b : balls) { /* ... */ } // Protected access to
        balls
4     score1++; // Protected access to score
5     // CRITICAL SECTION END
6 }

```

Listing 6: Critical Section: Board.updateState()

5.2.2 2. Deadlock Freedom

Property: No circular wait-for cycles among threads.

Lock Hierarchy:

Lock Order	Acquisition Pattern
1. Board	Always acquired first (main game state)
2. Bot Monitor	Acquired only from within Bot thread
3. RenderSync	Acquired from Update thread only

Table 2: Lock Hierarchy for Deadlock Prevention

Analysis:

- Bot thread never acquires Board after waiting on botMonitor
- No wait chains among Update, Bot, and EDT threads
- Circular dependencies are structurally impossible

5.2.3 3. Liveness (Game Termination)

Property: Game always terminates within bounded time steps.

Termination Conditions:

1. **Score-based:** All small balls scored or removed
2. **Player-based:** A player's ball enters a hole

3. User-initiated: Window closed or game stopped

JPF Verification:

```

1 public void testGameTermination() {
2     // JPF will explore all possible execution paths
3     // and verify that all paths eventually reach:
4     assert !isRunning() : "Game must terminate";
5     assert gameOverFlag : "Game-over state must be set";
6 }

```

Listing 7: Termination Assertion

5.3 Race Condition Detection

5.3.1 Synchronized Field Accesses

JPF verifies that all shared mutable fields are protected:

```

1 private List<Ball> balls;           // Protected by synchronized
   methods
2 private int score1;                 // Protected by synchronized
   methods
3 private boolean running;           // Protected by synchronized
   getter/setter
4 private boolean isGameOver;         // Protected by synchronized
   methods

```

Listing 8: Synchronized Field Protection

5.3.2 Temporal Assertions

Critical invariants verified during execution:

```

1 // Invariant 1: Score consistency
2 assert score1 >= 0 && score2 >= 0 : "Scores are non-negative";
3
4 // Invariant 2: Ball list validity
5 assert balls != null : "Ball list always exists";
6 assert balls.stream().allMatch(b -> b != null) :
7     "No null balls in list";
8
9 // Invariant 3: Game state consistency
10 assert !(isGameOver && !running) :
11     "Game-over implies not running";

```

Listing 9: Invariants Checked by JPF

5.4 Model Checking Results

5.4.1 Test Configuration: MainJPF

The MainJPF class provides a minimal test case for JPF verification:

```
1 public class MainJPF {
2     public static void main(String[] args) {
3         // Minimal board configuration for JPF
4         Board board = new Board();
5         board.init(new MinimalBoardConf());
6
7         GameController controller =
8             new GameController(board, null, null);
9
10        // JPF explores all possible interleavings
11        controller.startGame();
12
13        // Verification terminates after bounded steps
14        try {
15            Thread.sleep(100);
16        } catch (InterruptedException e) {
17            // Expected: JPF may interrupt
18        }
19
20        controller.stopGame();
21    }
22 }
```

Listing 10: MainJPF Verification Driver

6 Conclusion

The concurrent implementation of the Pool Game successfully addresses the performance bottlenecks of the sequential version through:

1. **Multi-threaded Architecture:** Separates physics computation, bot AI, and rendering into independent threads
2. **Monitor-Based Synchronization:** Provides clean abstraction for thread coordination
3. **Lock-Free Rendering:** RenderSync monitor prevents view corruption without blocking updates
4. **Formal Verification:** JPF proves correctness properties (deadlock-freedom, race-freedom) for minimal configurations

References

References

- [1] NASA Formal Methods Program, *Java Path Finder - Model Checker for Java Programs*, <https://javapathfinder.github.io/>

- [2] J.L. Peterson, *Petri Nets*, *ACM Computing Surveys*, vol. 9, no. 3, 1977.
- [3] C.A.R. Hoare, *Monitors: An Operating System Structuring Concept*, *Communications of the ACM*, 1974.
- [4] B. Goetz et al., *Java Concurrency in Practice*, Addison-Wesley, 2006.

A Code Listings

A.1 GameController Main Loop

```
1 private void updateLoop() {
2     long lastUpdateTime = System.currentTimeMillis();
3     int nFrames = 0;
4     long t0 = System.currentTimeMillis();
5
6     while (isRunning()) {
7         long elapsed = System.currentTimeMillis() -
8             lastUpdateTime;
9         lastUpdateTime = System.currentTimeMillis();
10
11         board.updateState(elapsed);
12
13         nFrames++;
14         viewModel.update(board, framePerSec);
15         view.render();
16
17         if (board.isGameOver()) {
18             setRunning(false);
19         }
20     }
```

Listing 11: Update Loop Implementation